

1. REST Naming Conventions & API Design

REST API design garda URLs (Endpoints) clear, predictable, ra resource-based hunaiparchha.

- **Rule 1: Always use Plural Nouns (बहुवचन):** URL ma kahilyai pani verbs (actions) jastai `/getUser` ba `/deleteProduct` nalekhne. Tyas ko thau ma HTTP methods use garne.
- **Rule 2: Lowercase strictly use garne:** `/api/productDetails` bhanda `/api/product-details` standard ho.

Action (Verb)	Wrong (Non-REST)	Correct REST Design	HTTP Method
Get All Products	<code>/api/getAllProducts</code>	<code>/api/products</code>	GET
Get Single Product	<code>/api/getProduct/1</code>	<code>/api/products/1</code>	GET
Create Product	<code>/api/createProduct</code>	<code>/api/products</code>	POST
Update Product	<code>/api/updateProduct/1</code>	<code>/api/products/1</code>	PUT / PATCH
Delete Product	<code>/api/deleteProduct/1</code>	<code>/api/products/1</code>	DELETE

2. API Versioning

Bhabishya ma frontend app updates huda backend code break nahos bhanera hami URL mai **Versioning** use garchhaũ.

- **Format:** `/api/v1/products`
- Jaba pachhadi gayera code ma thulo change aauchha, hami naya endpoints `/api/v2/products` banauchhaũ, jasle garda purano mobile app chalirakhya user haru ko app crash hudaina.

3. Professional Response Structure

Client (React/Android) lai data pathauda random format ma hoina, fixed standard envelope structure ma pathaunuparchha:

json

```
{
  "success": true,
  "message": "Products fetched successfully",
  "meta": {
    "currentPage": 1,
    "limit": 10,
    "totalItems": 150
  },
  "data": [ ... ]
}
```

Use code with caution.

4. Advanced Features (Pagination, Filtering, Sorting, Searching)

Jaba database ma lakhau data hunchha, sabae data 1 choti mai user lai pathauda server crash hunchha. Tyasko lagi hami query parameters (`req.query`) use garera advanced filtering pipeline banauchhaũ.

 **Full Combined Implementation Code (`controllers/productController.js`):**

Yo single function bhitra professional logic kasari build hunchha, setup herũ:

javascript

```
// Dummy Database array mimicking a real collection
const allProducts = [
  { id: 1, name: "Apple iPhone 15", category: "electronics", price: 120000 },
  { id: 2, name: "Samsung S24 Ultra", category: "electronics", price: 145000 },
  { id: 3, name: "Nike Air Max", category: "shoes", price: 15000 },
  { id: 4, name: "Adidas Ultraboost", category: "shoes", price: 18000 },
  { id: 5, name: "Dell XPS 13", category: "electronics", price: 160000 }
];

const getProductsAdvanced = (req, res) => {
  try {
    let results = [...allProducts];

    // 🔍 A. SEARCHING (URL: ?search=iphone)
    if (req.query.search) {
      const searchKeyword = req.query.search.toLowerCase();
      results = results.filter(p =>
        p.name.toLowerCase().includes(searchKeyword));
    }
  }
}
```

```

// 🧩 B. FILTERING (URL: ?category=electronics)
if (req.query.category) {
    results = results.filter(p => p.category ===
req.query.category);
}

// ⬇ C. SORTING (URL: ?sortBy=price&order=desc)
if (req.query.sortBy === 'price') {
    const order = req.query.order === 'desc' ? -1 : 1;
    results.sort((a, b) => (a.price - b.price) * order);
}

// 📄 D. PAGINATION (URL: ?page=1&limit=2)
const totalItems = results.length;
const page = parseInt(req.query.page) || 1;
const limit = parseInt(req.query.limit) || 2;

// Math Calculation for offset slicing
const startIndex = (page - 1) * limit;
const endIndex = page * limit;
results = results.slice(startIndex, endIndex);

// 📦 Standard Response Output Delivery
res.status(200).json({
    success: true,
    message: "Products filtered and matching results
successfully",
    meta: {
        totalRecords: totalItems,
        currentPage: page,
        totalPages: Math.ceil(totalItems / limit),
        limitPerPage: limit
    },
    data: results
});

} catch (error) {
    res.status(500).json({ success: false, message: "Internal server
error" });
}
};

module.exports = { getProductsAdvanced };
Use code with caution.

```

🔧 5. Complete End-to-End CRUD Operations (The Core Router Setup)

Aba hami purei design pattern merge garera clean routes architecture build garchhaũ:

`routes/api.js` (Routing Controller Map)

javascript

```
const express = require('express');
const router = express.Router();
const { getProductsAdvanced } =
  require('../controllers/productController');

// Version 1 endpoints mapping standard
router.get('/v1/products', getProductsAdvanced);

// Simulated standard CRUD routes controllers assignment
router.post('/v1/products', (req, res) => {
  res.status(201).json({ success: true, message: "Resource successfully
  created" });
});

router.put('/v1/products/:id', (req, res) => {
  res.status(200).json({ success: true, message: `Resource with ID
  ${req.params.id} completely updated` });
});

router.delete('/v1/products/:id', (req, res) => {
  res.status(200).json({ success: true, message: `Resource with ID
  ${req.params.id} purged from system` });
});

module.exports = router;
Use code with caution.
```

6. API Testing (Postman / Thunder Client)

API logic codes banayesi code test garna frontend design banna bhanda agadi **Postman** ba VS Code extension **Thunder Client** use hunchha.

How to test our advanced endpoint inside Postman:

1. **Request Type:** Choose dropdown setup inside Postman to `GET`.
2. **Base URL Paste:** Input `http://localhost:3000/api/v1/products`
3. **Testing Features via Params Tab:** Postman ko "Params" configuration grid area ma key-values thapdai try garne:
 - o search → Nike

- `page` → 1
 - `limit` → 2
 - `sortBy` → `price` | `order` → `desc`
4. **Send Button click:** Tapaile real-time pipeline format processing results response metadata wrapper sahiti console visualization ma dekhnu hunchha!
-

Essential Phase Review Checklist:

- **REST Architecture** ma endpoints simple nouns plural tracking systems format hunaiparchha.
- **Pagination** query parameters slicing runtime server storage loading safe rakhna unmatched solution ho.
- Always preserve tracking systems utilizing proper **HTTP Status Response Codes** matching operations (200 for OK, 201 for Created, 404 for Not Found).

REST API architectures design concepts blueprint complete bhayo. Aba real persistent hardware cluster pipeline ma records operations write garna hami **Phase 8: Databases integration & Mongoose Object Data Modeling** thalna sakchhaũ.